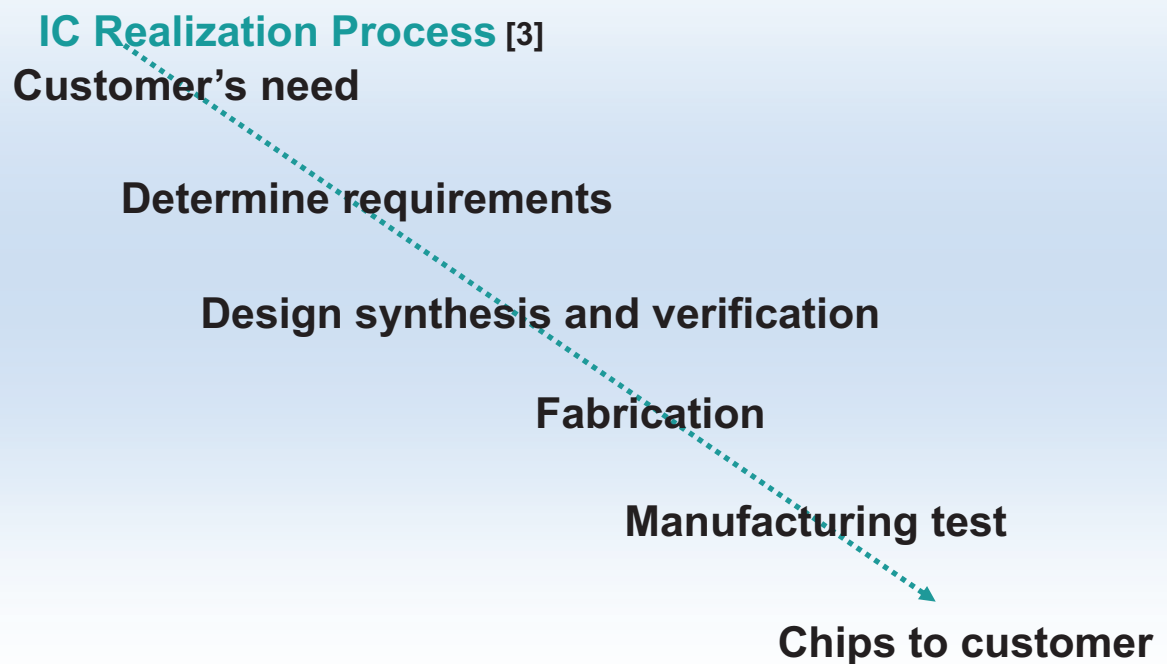


# IC Testing

1. Manufacturing Test Principles
2. Design for Testability (DFT)

# Intro to Testing



## Verification vs. Testing

- **Verification:** Predictive analysis to ensure that the synthesized design, when manufactured, will perform the given I/O function.
- **Test:** A manufacturing step that ensures that the physical device, manufactured from the synthesized design, has no manufacturing defect. [3]

## Verification vs. Testing

### Verification

- Verifies correctness of design.
- Performed by simulation, hardware emulation, or formal methods.
- Performed once prior to manufacturing.
- Responsible for quality of design.

### Test

- Verifies correctness of manufactured hardware.
- Two-part process:
  - 1. Test pattern generation: process executed once during design
  - 2. Test application: electrical tests applied to hardware
- Test application performed on every manufactured device.
- Responsible for quality of devices. [3]

# Ideal Tests

- Ideal tests detect all defects produced in the manufacturing process.
- Ideal tests pass all functionally good devices.
- Very large numbers and varieties of possible defects need to be tested.

# Real Tests

- Based on fault models
  - Incomplete coverage of modeled faults due to high complexity.
  - Some good chips are rejected. The fraction (or percentage) of such chips is called the *yield loss*.
  - Some bad chips pass tests. The fraction (or percentage) of bad chips among all passing chips is called the *defect level*.
- [3]

## Roles of Testing

- Detection: Determination whether or not the *device under test* (DUT) has some fault.
- Diagnosis: Identification of a specific fault that is present on DUT.
- *Failure mode analysis* (FMA): Determination of manufacturing process errors that may have caused defects on the DUT. [3]

## Testing

- A speck of dust on a wafer is sufficient to kill chip
- *Yield* of any chip manufacturing is  $< 100\%$ 
  - Must test chips after manufacturing before delivery to customers to only ship good parts
- Manufacturing testers are very expensive
  - Minimize time on tester
  - Careful selection of *test vectors*



[1]

# Defect Level

- *Defect level* (DL) is the ratio of faulty chips among the chips that pass tests.
- DL is measured as *parts per million* (ppm).
- DL is a measure of the effectiveness of tests.
- DL is a quantitative measure of the manufactured product quality. For commercial VLSI chips a DL greater than 500 ppm is considered unacceptable.

[3]

customer  
requirement

# Defect Level

calculate your fault coverage  
of the manufacturing test  
needed to satisfy the customer  
requirement on the defect  
level

$$DL(T) = 1 - Y^{(1-T)}$$

Where  $T$  is the fault coverage of tests and  $Y$  is the manufacturing yield

## Definition of **Yield**:

Ratio of the number of functional chips per wafer to the number of total chips per wafer

ideal manufacturing process  $Y=100\%$   
 -->  $DL=0$   
 $Y=90\%=0.9$  ideal test  $T=100\%$   
 -->  $DL=0$   
 $Y=0.9$   $T=0.995$   
 -->  $DL=527\text{ppm}$

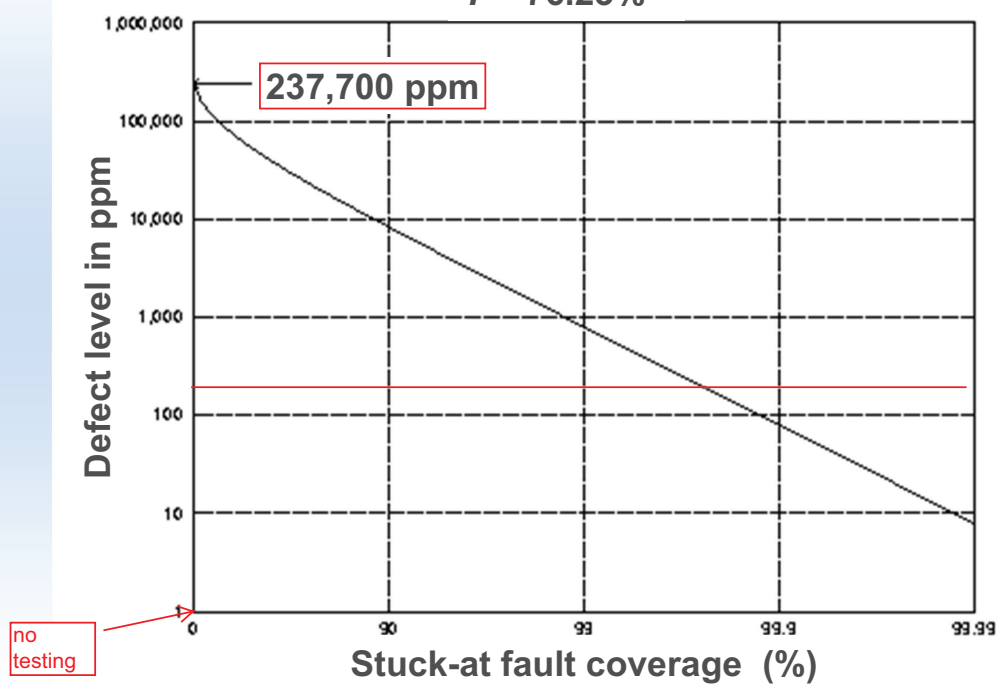
# Fault Coverage T

Required T for DL=200ppm as a function of the yield:

|      |       |       |      |      |
|------|-------|-------|------|------|
| Y(%) | 10    | 50    | 90   | 95   |
| T(%) | 99.99 | 99.97 | 99.8 | 99.6 |

## Defect Level vs. Fault Coverage

$Y = 76.23\%$



[3]

## Chip Cost

- A manufacturing defect is a finite chip area with electrically malfunctioning circuitry caused by errors in the fabrication process.
- A chip with no manufacturing defect is called a good chip.
- Fraction (or percentage) of good chips produced in a manufacturing process is called the *yield*. Yield is denoted by symbol  $Y$ .
- Cost of a chip=

$$\frac{\text{Cost of fabricating and testing a wafer}}{\text{Yield} \times \text{Number of chips on the wafer}}$$

[3]

## Summary of Definitions

- Yield drops as chip area increases; low yield means high cost.
- Fault coverage measures the test quality.
- Defect level (DL) is a measure of chip quality.
- DL can be determined by an analysis of test data.
- For high quality:  $DL < 500 \text{ ppm}$ , fault coverage  $\sim 99\%$

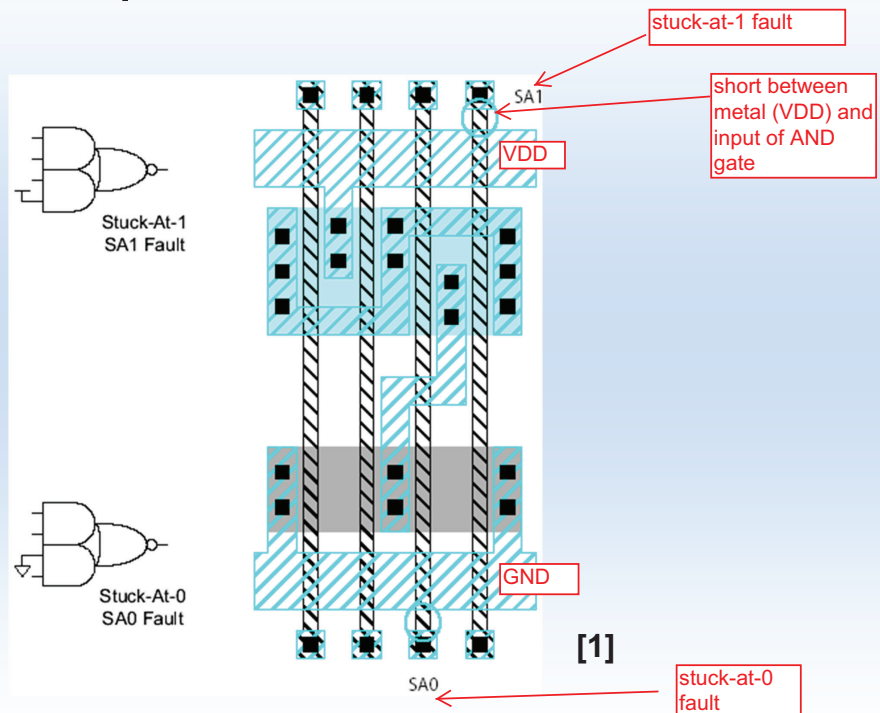
# Fault Modeling

- How does a chip fail?
  - Usually failures are shorts between two conductors or opens in a conductor
  - This can cause very complicated behavior
- A simpler model: *Stuck-At*
  - Assume a failure causes a node to be “stuck-at” 0 or 1, i.e. shorted to GND or  $V_{DD}$
  - Not quite true, but works well in practice

# Stuck-At Fault Model

- This model assumes that only one node is faulty!
- Stuck-at-0 represents a node that is permanently LOW regardless of the signals that control this node!
- Stuck-at-1 represents a node that is permanently HIGH regardless of the signals that control this node!

## Example: Stuck-At Fault



Prof. Schumann

VLSI Design and Testing

287

## Controllability & Observability

- *Controllability*: ability to set ("1") and reset ("0") every internal node by setting the primary inputs to specific values
- *Observability*: ability to observe the logical state of any node within the circuit at the primary outputs
- *Primary inputs*: inputs of the circuit that can be directly controlled by the test environment (e.g. input pads)
- *Primary output*: output of the circuit that can be directly observed by the test environment (e.g. output pads)

Prof. Schumann

VLSI Design and Testing

288

## Controllability & Observability

- Combinational logic is usually easy to observe and control
- Finite state machines can be very difficult, requiring many cycles to enter desired state
  - Especially if state transition diagram is not known to the test engineer

## Test Pattern Generation

- Manufacturing test ideally would check every node in the circuit to prove it is not stuck.
- Apply the smallest sequence of test vectors necessary to prove each node is not stuck.

How to check a node for a possible stuck-at fault?

# Test Pattern Generation

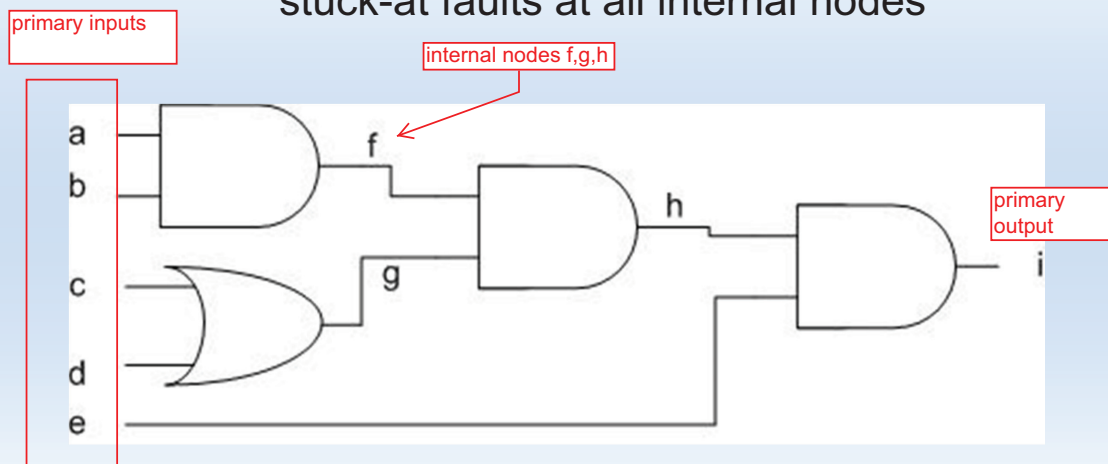
Algorithm to detect a stuck-at fault:

1. Control the faulty node to the opposite of the stuck-at value by applying data at the primary inputs!
2. Make the nodes fault observable at a primary output by controlling the value of all other nodes affecting the output response, so that the faulty node is the controlling node!

Aim: Generate a test pattern that can detect all possible stuck-at faults at all internal nodes

# Test Pattern Generation

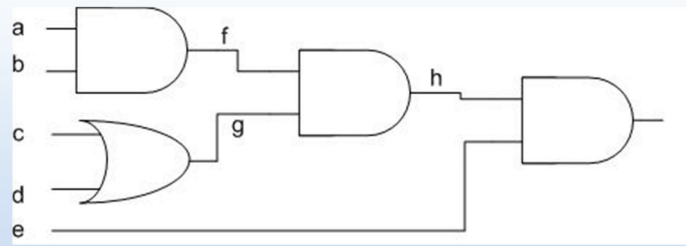
Example: test pattern that detects all possible stuck-at faults at all internal nodes



we need to generate a test pattern which can detect ST0 and ST1 fault on each of the 3 internal nodes! in total 6 test vectors!  
 check on possible ST0 fault on node f:  
 1. force f to 1 --> a=b=1  
 2. g=1, e=1 --> (c,d)=(1,0) or (0,1) or (1,1)  
 3 possible test vectors (a,b,c,d,e)=(1,1,1,0,1) or (1,1,0,1,1) or (1,1,1,1,1)  
 Prof. Schumann

check on a possible ST1 on node f:  
 1. force f to 0 --> a=0 and b=0 or a=0 and b=1 or a=1 and b=0  
 2. g=1 --> c=1 and d=1 or c=0 and d=1 or c=1 and d=0  
 e=1  
 9 possible test vectors (a,b,c,d,e)=.....

# Test Pattern Generation



**node h: check on stuck-at 0 fault**

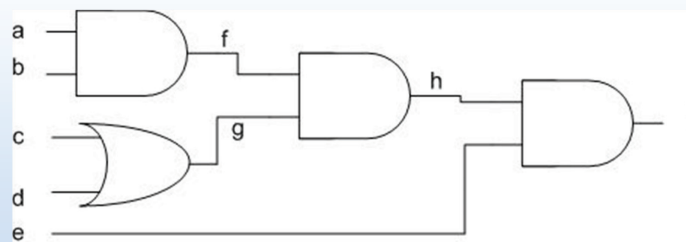
- ①. control h to '1':  $h = f \cdot g \equiv 1$ 
    - $g=1 \wedge f=1$
    - $f = a \cdot b \equiv 1$
    - $a=1 \wedge b=1$
    - $g = c + d \equiv 1$
    - $c=1 \wedge d=0$  or
    - $c=0 \wedge d=1$  or
    - $c=1 \wedge d=1$
  - ②. observe fault on node i
    - $i = h \cdot e$
    - $e=1$  (only node h controls i)
- $\{a,b,c,d\} = \{1,1,1,0\}$   
 or  $\{1,1,0,1\}$   
 or  $\{1,1,1,1\}$
- $\{a,b,c,d,e\} = \{1,1,1,0,1\}$   
 or  $\{1,1,0,1,1\}$   
 or  $\{1,1,1,1,1\}$   
 (1)

Prof. Schumann

VLSI Design and Testing

293

# Test Pattern Generation



**node h: check on stuck-at 1 fault**

- ①. control h to '0':  $h = f \cdot g \equiv 0$ 
    - $f=0 \wedge g=0$  or
    - $f=1 \wedge g=0$  or
    - $f=0 \wedge g=1$
  - ②. observe fault on node i
    - $e=1$
- $\{a,b,c,d\} = \{0,0,x,x\}$   
 or  $\{0,1,x,x\}$   
 or  $\{1,0,x,x\}$   
 or  $\{1,1,0,0\}$
- $\{a,b,c,d,e\} = \{0,0,x,x,1\}$   
 or  $\{0,1,x,x,1\}$   
 or  $\{1,0,x,x,1\}$   
 or  $\{1,1,0,0,1\}$   
 (2)

Prof. Schumann

VLSI Design and Testing

294

# Test Pattern Generation

node f: stuck-at 0

control f to '1'  
 $\rightarrow \{a,b\} = \{1,1\}$

observe on node i:  $g=1, e=1$   
 $\rightarrow \{c,d,e\} = \{1,0,1\}$   
 $\{0,1,1\}$   
 $\{1,1,1\}$

$\rightarrow \{a,b,c,d,e\} = \{1,1,1,0,1\}$   
 or  $\{1,1,0,1,1\}$   
 or  $\{1,1,1,1,1\}$   
 (3)

node f: stuck-at 1

control f to '0'  
 $\rightarrow \{a,b\} = \{0,0\}$   
 $\{0,1\}$   
 $\{1,0\}$   
 observe on node i:  $g=1$   
 $\rightarrow \{c,d\} = \{0,1\}$   
 $\{1,0\}$   
 $\{1,1\}$

$e = 1$

$\rightarrow \{a,b,c,d,e\} = \{0,0,0,1,1\}$   
 or  $\{0,0,1,0,1\}$   
 or  $\{0,0,1,1,1\}$   
 or  $\{0,1,0,1,1\}$   
 or  $\{0,1,1,0,1\}$   
 or  $\{0,1,1,1,1\}$   
 or  $\{1,0,0,1,1\}$   
 or  $\{1,0,1,0,1\}$   
 or  $\{1,0,1,1,1\}$   
 (4)

node g: stuck-at 0

control g to '1'  
 $\rightarrow \{c,d\} = \{0,1\}$   
 $\{1,0\}$   
 $\{1,1\}$

Observe on node i:  $f=1$   
 $\rightarrow \{a,b\} = \{1,1\}$   
 $e = 1$

$\rightarrow \{a,b,c,d,e\} = \{1,1,1,0,1\}$   
 or  $\{1,1,0,1,1\}$   
 or  $\{1,1,1,1,1\}$   
 (5)

Prof. Schumann

VLSI Design and Testing

295

# Test Pattern Generation

node g: stuck-at 1

control g to '0'  
 $\rightarrow \{c,d\} = \{0,0\}$   
 observe on node i:  $f=1$   
 $\{a,b\} = \{1,1\}$   
 $e = 1$

$\rightarrow \{a,b,c,d,e\} = \{1,1,0,0,1\}$  (6)

smallest sequence of 3 test vectors necessary:

|                       |                 |
|-----------------------|-----------------|
| For stuck-at (6)+(2): | $\{1,1,0,0,1\}$ |
| (1)+(3)+(5):          | $\{1,1,0,1,1\}$ |
| (4):                  | $\{0,0,1,0,1\}$ |

Prof. Schumann

VLSI Design and Testing

296

## Design for Testability (DFT)

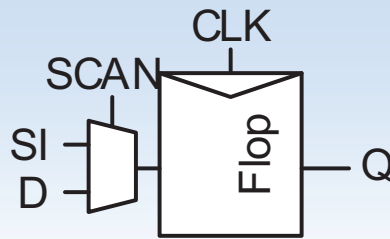
- Design the chip to increase observability and controllability.
- Good observability and controllability reduces cost of testing because they allow high fault coverage with relatively few test vectors.
- DFT can be categorized in two main approaches:
  - Scan Design
  - Built-in Self-Test (BIST)

## Scan Design

- If each flip-flop could be observed and controlled, test problem reduces to testing combinational logic between registers.
- In designs with scan, the flip-flops operate in one of two modes:
  - In normal mode flip-flops behave as usual
  - In scan mode they are connected to form a shift register called a scan chain spanning the whole chip

# Scan Design

- Convert each flip-flop to a *scan register*
  - Only costs one extra multiplexer
- Normal mode (SCAN=0): data is loaded from D pin
- Scan mode (SCAN=1): test data is loaded from SI pin



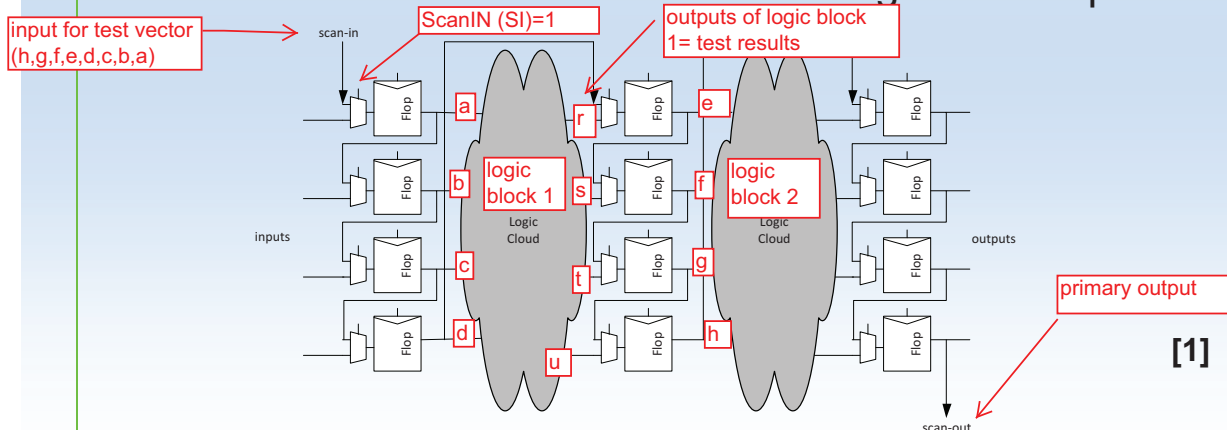
scan register

[1]

# Scan Design

In scan mode the scan registers are connected to form a *scan chain*:

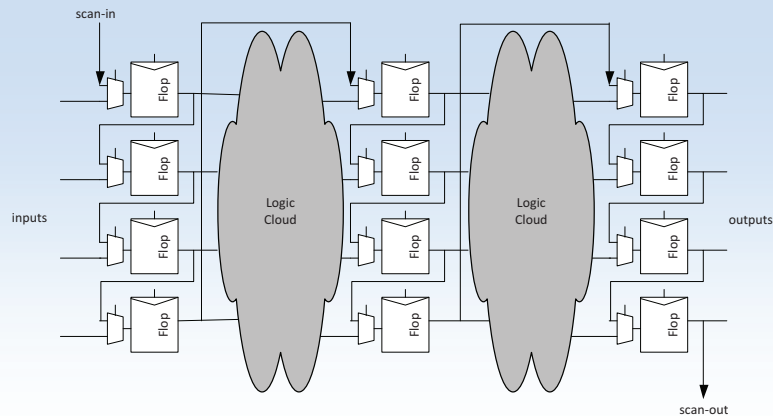
- SI pin is connected to previous register Q output
- Test pattern can be scanned in using scan-in pin
- Stored data can be scanned out using scan-out pin



# Scan Design

3-steps testing procedure using Scan Design:

1. Load the scan chain: SCAN signal is activated, apply clock signal for eight clock cycles to load the first two ranks of 4-bit scan registers with data using scan-in pin. This is the test pattern for the two logic clouds.



[1]

Prof. Schumann

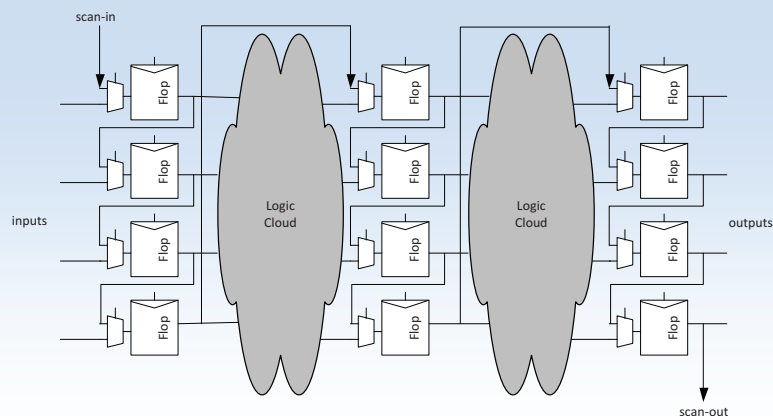
VLSI Design and Testing

301

# Scan Design

Testing procedure using Scan Design:

2. Operate the circuit in normal mode: SCAN signal is deactivated, run the circuit for one clock cycle



[1]

Prof. Schumann

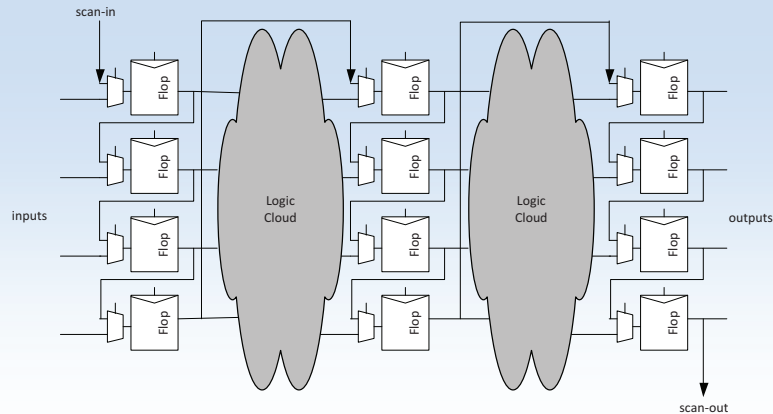
VLSI Design and Testing

302

# Scan Design

Testing procedure using Scan Design:

3. Read the stored data out: SCAN signal is activated, apply clock signal for eight clock cycles to read the stored data out using scan-out pin.



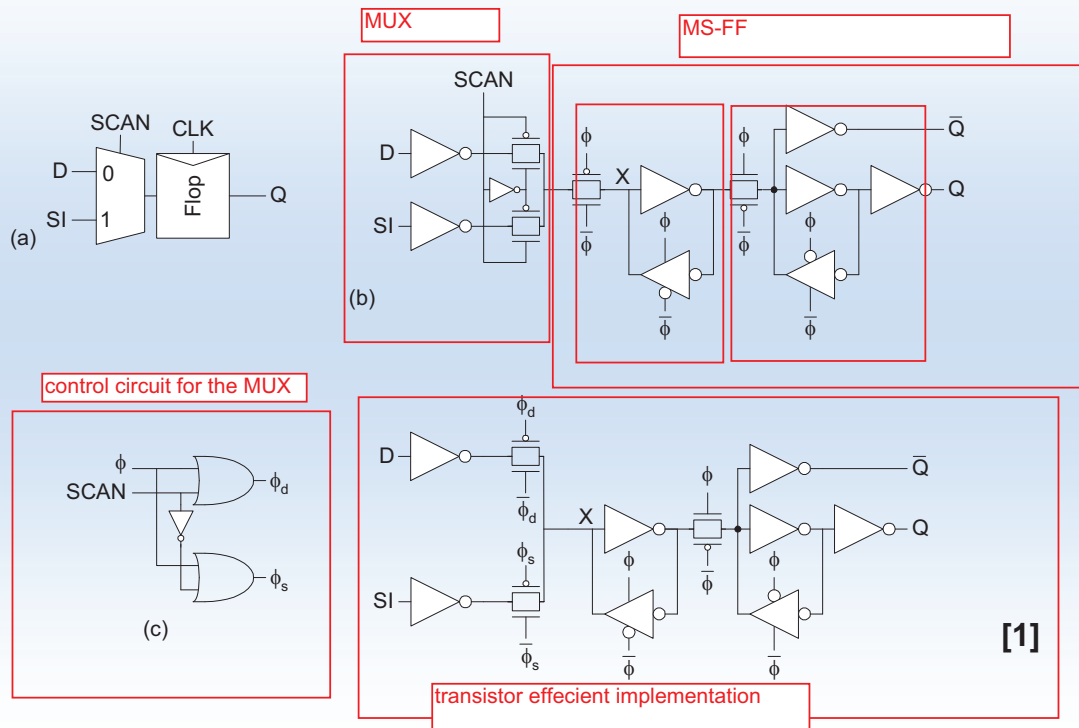
[1]

Prof. Schumann

VLSI Design and Testing

303

# Scan-able Flip-Flops



[1]

Prof. Schumann

VLSI Design and Testing

304

# ATPG

- ❑ Test pattern generation is tedious
- ❑ Automatic Test Pattern Generation (ATPG) tools produce a set of vectors for each block of combinational logic
- ❑ Scan chains are used to control and observe the blocks

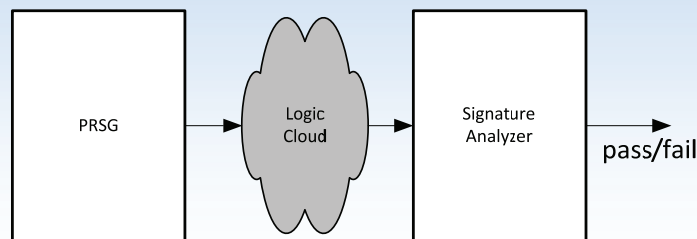
[1]

# Built-in Self-test

- Built-in self-test (BIST) lets circuits test themselves
  - Circuit itself generates a test pattern
  - Circuit itself decides whether test result is correct or not
- Built-in self-test reduces the test time
- Test time is often more expensive than additional chip area

## Built-in Self-test

- Built-in self-test lets blocks test themselves
  - Generate pseudo-random inputs to comb. logic by Pseudo Random Sequence Generator (PRSG)
  - Output data is compressed into a short string of bits called the *signature*. This signature is compared with the signature of a correctly working device. Compression and comparison is done within the Signature Analyzer.



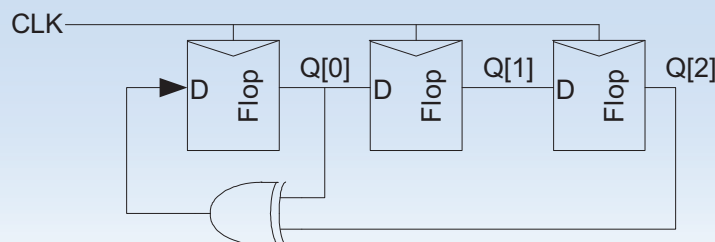
Prof. Schumann

VLSI Design and Testing

307

## Pseudo Random Sequence Generator

- *Linear Feedback Shift Register* (LFSR) is often used to generate pseudo-random test patterns. It is a shift register with two or more flip-flop outputs are XOR'ed together and feedback into the first flip-flop.



PRSG: 3-bit LFSR

[1]

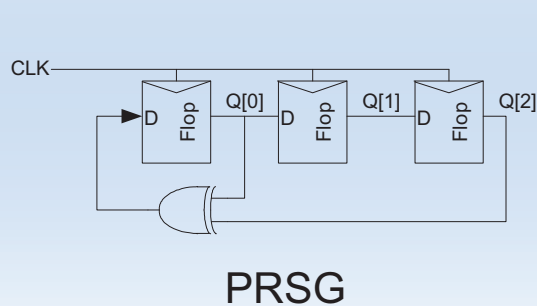
Prof. Schumann

VLSI Design and Testing

308

# Pseudo Random Sequence Generator

- With  $n$ -bit LFSR it is possible to generate  $2^n - 1$  different bit patterns.

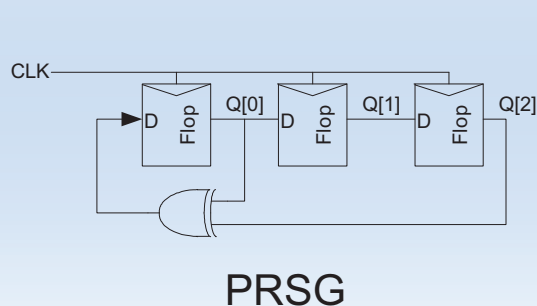


| Step | Q <span style="border: 1px solid red; padding: 2px;">Q2 Q1 Q0</span> |
|------|----------------------------------------------------------------------|
| 0    | 111                                                                  |
| 1    | <span style="border: 1px solid red; padding: 2px;">110</span>        |
| 2    | <span style="border: 1px solid red; padding: 2px;">101</span>        |
| 3    |                                                                      |
| 4    |                                                                      |
| 5    |                                                                      |
| 6    |                                                                      |
| 7    |                                                                      |

[1]

# Pseudo Random Sequence Generator

- With  $n$ -bit LFSR it is possible to generate  $2^n - 1$  different bit patterns.



| Step | Q             |
|------|---------------|
| 0    | 111           |
| 1    | 110           |
| 2    | 101           |
| 3    | 010           |
| 4    | 100           |
| 5    | 001           |
| 6    | 011           |
| 7    | 111 (repeats) |

[1]

# Pseudo Random Sequence Generator

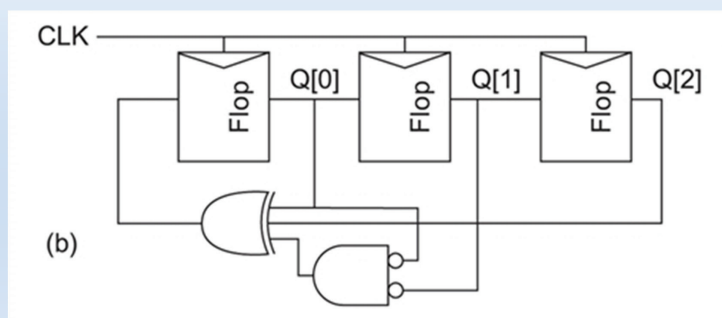
- With n-bit LFSR it is possible to generate  $2^n - 1$  different bit patterns. All patterns can be generated except for all 0's! The patterns have certain randomness properties: PRSG

| n     | feedback                                     |
|-------|----------------------------------------------|
| 4,6,7 | $Q_0 \oplus Q_{n-1}$                         |
| 5     | $Q_1 \oplus Q_4$                             |
| 8     | $Q_1 \oplus Q_2 \oplus Q_3 \oplus Q_7$       |
| 12    | $Q_0 \oplus Q_3 \oplus Q_5 \oplus Q_{11}$    |
| 14,16 | $Q_2 \oplus Q_3 \oplus Q_4 \oplus Q_{n-1}$   |
| 24    | $Q_0 \oplus Q_1 \oplus Q_6 \oplus Q_{23}$    |
| 32    | $Q_0 \oplus Q_1 \oplus Q_{21} \oplus Q_{31}$ |

Feedback for maximum length LFSR sequence

# Pseudo Random Sequence Generator

- If the all-0s test is required, an n-bit LFSR can be modified by adding an NOR-gate with n-1 inputs.



notA and notB= nor (A,B)

When in state 001, the next is state 000

[1]

# Signature Analyzer

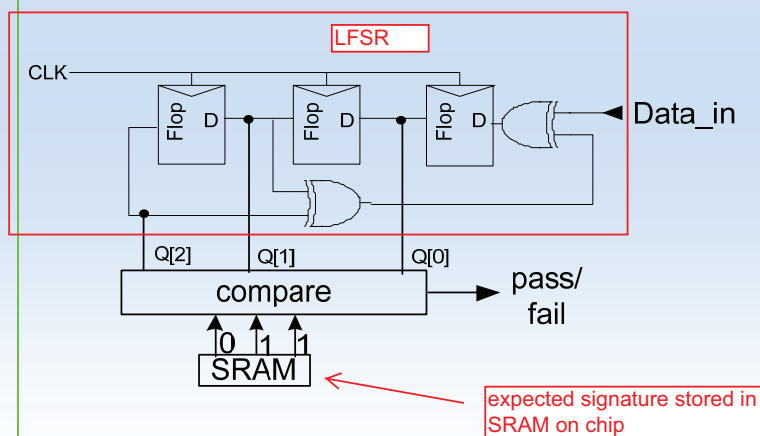
- Def. signature: compressed output response of a circuit under test
- Def. signature analyzer: block which generates the signature and performs the comparison with the expected signature

# Signature Analyzer

Generator of signature: *Linear Feedback Shift Register*

Expected signature: store in *SRAM*

Comparison: *AND-Gate*



Example: correct output response

| Step | Data_in | Q <sub>2</sub> Q <sub>1</sub> Q <sub>0</sub> |
|------|---------|----------------------------------------------|
| 0    | 10101   | 000                                          |
| 1    | 0101    | 001                                          |
| 2    | 101     | 010                                          |
| 3    | 01      | 100                                          |
| 4    | 1       | 001                                          |
| 5    |         | 011 (signature as expected)                  |

pass

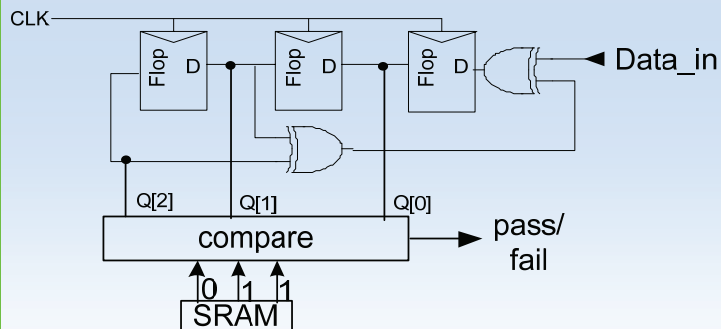
# Signature Analyzer

Generator of signature: *Linear Feedback Shift Register*

Expected signature: store in *SRAM*

Comparison: *AND-Gate*

Example: faulty output response



| Step | Data_in | Q <sub>2</sub> Q <sub>1</sub> Q <sub>0</sub> |
|------|---------|----------------------------------------------|
| 0    | 1001    | 000                                          |
| 1    | 0101    | 001                                          |
| 2    | 101     | 010                                          |
| 3    | 01      | 101                                          |
| 4    | 1       | 011                                          |
| 5    |         | 110 (signature false)                        |

fail

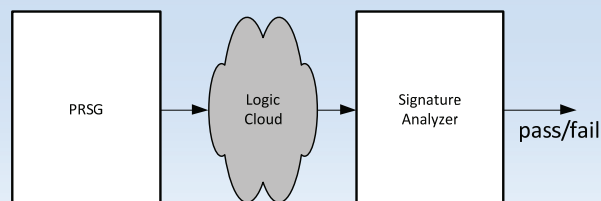
Prof. Schumann

VLSI Design and Testing

315

# Summary Testing

- Think about testing from the beginning of design
- Yield of any chip manufacturing is < 100%
- Design the chip to increase observability and controllability (e.g. scan chain)
- Consider Built-in Self-test (BIST)



Prof. Schumann

VLSI Design and Testing

316